

Project 2 (Linda Miao)

1. Schema and Query

Database **Linda_QO_Project2** has 3 tables: **Students** (StudentID, StudentName, Age, City), **Classes** (ClassID, ClassName, Semester), and **Enrollments** (EnrollmentID, StudentID, ClassID, Grade). Indexes: PK on each table (Clustered), IX_Students_City (Non-Clustered), IX_Enrollments_StudentID and IX_Enrollments_ClassID (Non-Clustered). The schema was fixed throughout — only row counts changed between situations.

The single query used for all 5 situations:

```
SELECT s.StudentName, c.ClassName, c.Semester,
       COUNT(e.EnrollmentID) AS TotalEnrollments
FROM Students s
JOIN Enrollments e ON s.StudentID = e.StudentID
JOIN Classes c   ON e.ClassID   = c.ClassID
WHERE s.City = 'Seattle'
GROUP BY s.StudentName, c.ClassName, c.Semester
```

2. Row Changes That Triggered Each Plan Flip

The table below records the exact row counts at each plan change point — the 'cliff' where a small increase in data volume caused the optimizer to change strategy entirely.

#	From Plan	To Plan	Row Count at Flip	Why It Changed
1	QP1 Nested Loop	QP2 Hash Match	500 students / 2,000 enrollments	Looping 2,000 rows per outer row exceeded cost of building hash table in memory
2	QP2 Hash Match	QP3 Merge Join	5,000 students / 30,000 enrollments	Sorted clustered index inputs made Merge Join cheaper than hashing at this scale

3	QP3 Merge Join	QP4 Parallel Plan	30,000 students / 400,000 enrollments	Data large enough that 6-core parallel execution reduced total cost significantly
4	QP4 Parallel	QP5 Parallel + Missing Index	50,000 students / 680,000 enrollments	Optimizer identified missing index on Enrollments(ClassID) with 38.58% impact

3. The 5 Query Plans — Graphical Showplans

QP1 — Nested Loops (5 students, 8 enrollments)

Situation 1 — Small Data (Natural Plan = QP1)

Rows in tables:

Students: 5

Classes: 3

Enrollments: 8

Join type: Nested Loops (Inner Join)

Access methods:

Students: Clustered Index Seek

Classes: Clustered Index Seek

Enrollments: Clustered Index Scan

Execution time: 0.000s

Rows returned: 6

Subtree cost: 0.0230052

Est. rows (SELECT): 0

Rows per execution: 4.43959

Size: 48 KB

Query 1: Query cost (relative to the batch): 100%

```

SELECT -- get studentName column from students table -- "s" is alias name for students s.StudentName, -- get classname column from classes table -- "c" is alias name for classes c.ClassName, -- get semester column from classes table c.Semester, -- count number of enrollments COUNT(e.EnrollmentID) AS TotalEnrollments -- start from students table -- s =

```

Query Statistics:

Statistic	Value
Cached plan size	48 KB
Estimated Operator Cost	0.00%
Degree of Parallelism	1
Estimated Subtree Cost	0.0230052
Memory Grant	1024 KB
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	4.43959

Statement:

```

SELECT
-- get studentName column from students table
-- "s" is alias name for students
s.StudentName,
-- get classname column from classes table
-- "c" is alias name for classes
c.ClassName,
-- get semester column from classes table
c.Semester,
-- count number of enrollments
COUNT(e.EnrollmentID) AS TotalEnrollments
-- start from students table
FROM Students s
-- s = alias (short name for students)
JOIN Enrollments e

```

QP2 — Hash Match (500 students, 2,000 enrollments)

Situation 2 — QP2 (Hash Match — plan flip!)

Rows in tables:

Students: 500

Classes: 3

Enrollments: 2000

Join type: Hash Match (Inner Join)

Access methods:

Students: Clustered Index Scan

Classes: Clustered Index Scan

Enrollments: Clustered Index Scan

Subtree cost: 0.172833

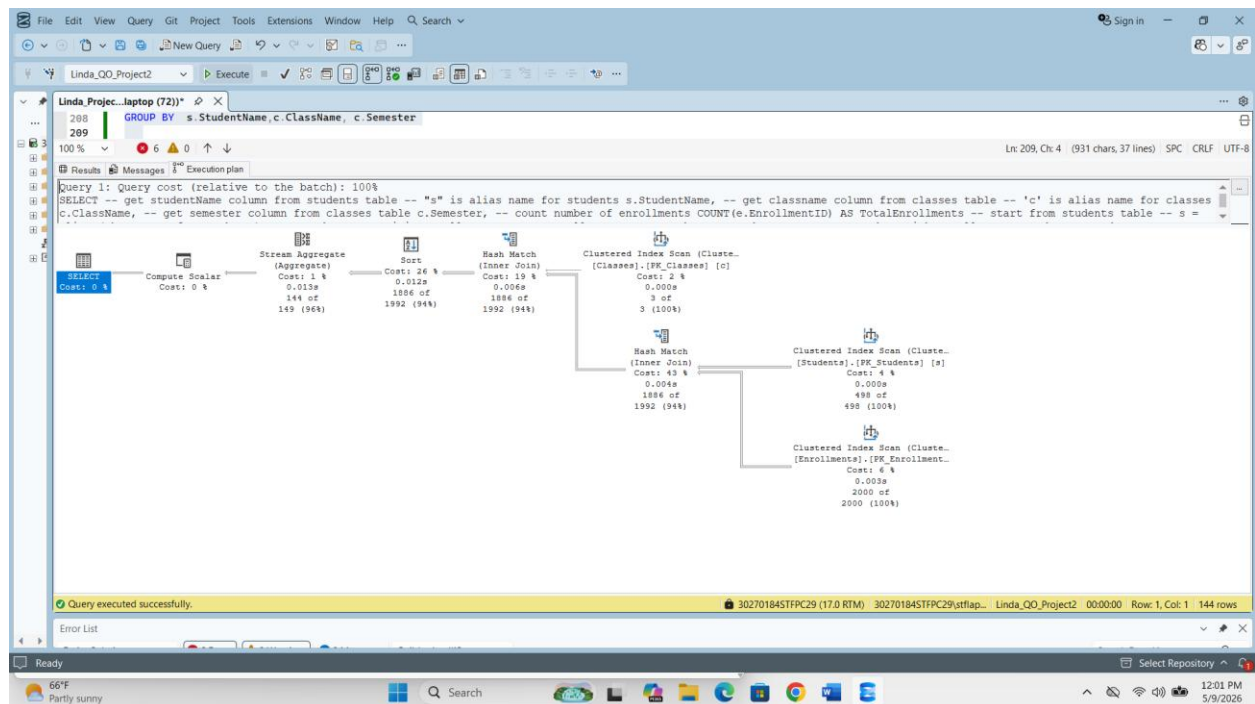
Memory grant: 3392 KB

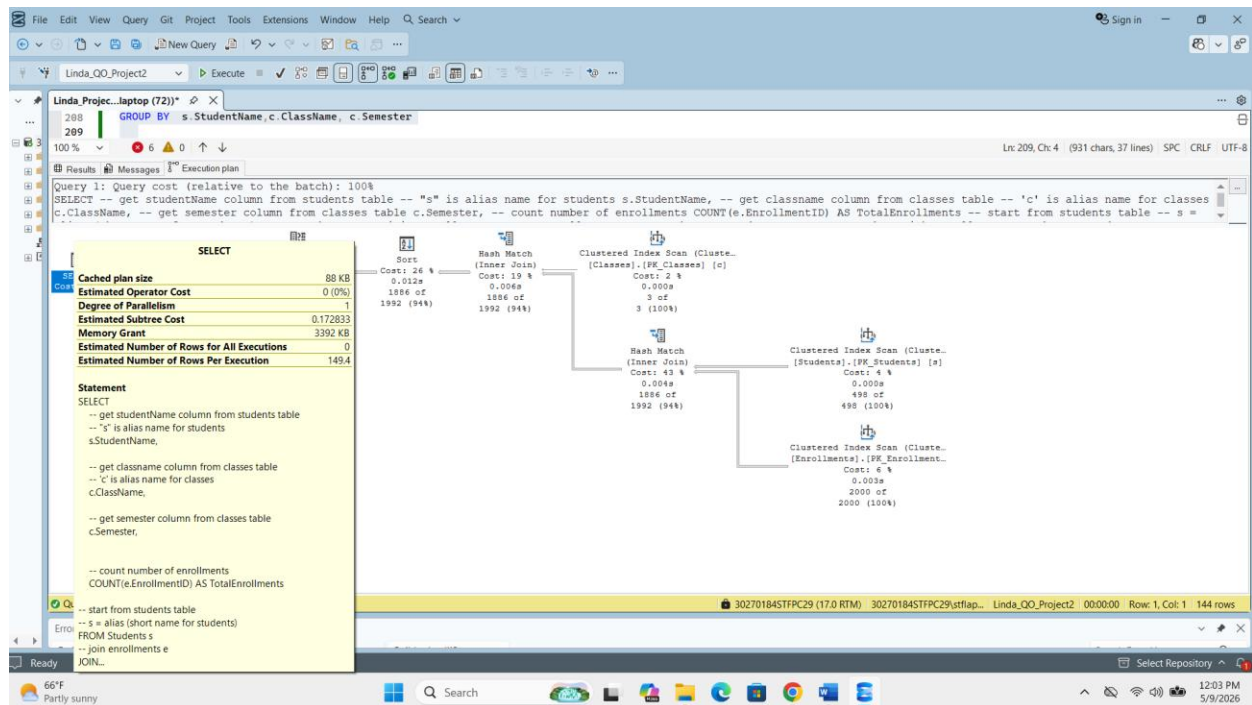
Rows per execution: 149.4

Size: 88 KB

Execution time: 0.000s

Rows returned: 144





QP3 — Merge Join (5,000 students, 30,000 enrollments)

Situation 3 — QP3 (Merge Join — second flip!)

Rows in tables:

Students: 5000

Classes: 3

Enrollments: 30000

Join type: Merge Join (Inner Join)

Access methods:

Students: Clustered Index Scan

Classes: Clustered Index Scan

Enrollments: Clustered Index Scan

Subtree cost: 1.05403

Memory grant: 4936 KB

Rows per execution: 1999.2

Size: 88 KB

Execution time: 0.000s

Rows returned: 5,994

The image displays two screenshots of a SQL IDE interface, likely Microsoft SQL Server Enterprise Manager, showing the execution of a query and its associated execution plan.

Top Screenshot: The query window shows the following SQL code:

```
-- group rows before counting( group by student, class, semester
GROUP BY s.StudentName, c.ClassName, c.Semester
```

The query results pane shows the following columns: `s.StudentName`, `c.ClassName`, `c.Semester`, and `Count(e.EnrollmentID) AS TotalEnrollments`. The results show 5,994 rows.

The execution plan pane shows the following steps:

- SELECT (Cost: 0.0000, Rows: 5994 of 1999 (299%))
- Compute Scalar (Cost: 0.0000, Rows: 5994 of 1999 (299%))
- Stream Aggregate (Aggregate) (Cost: 0.0000, Rows: 5994 of 1999 (299%))
- Sort (Cost: 0.0000, Rows: 5994 of 1999 (299%))
- Merge Join (Inner Join) (Cost: 2.0000, Rows: 5994 of 1999 (299%))
- Clustered Index Scan (Clustered Index Scan) (Cost: 3.0000, Rows: 4998 of 3998 (39%))
- Sort (Cost: 4.0000, Rows: 4000 of 2000 (300%))
- Hash Match (Aggregate) (Cost: 41.0000, Rows: 4000 of 2000 (300%))
- Hash Match (Inner Join) (Cost: 28.0000, Rows: 30000 of 30000 (100%))
- Clustered Index Scan (Clustered Index Scan) (Cost: 1.0000, Rows: 500 of 500 (100%))
- Clustered Index Scan (Clustered Index Scan) (Cost: 10.0000, Rows: 30000 of 30000 (100%))

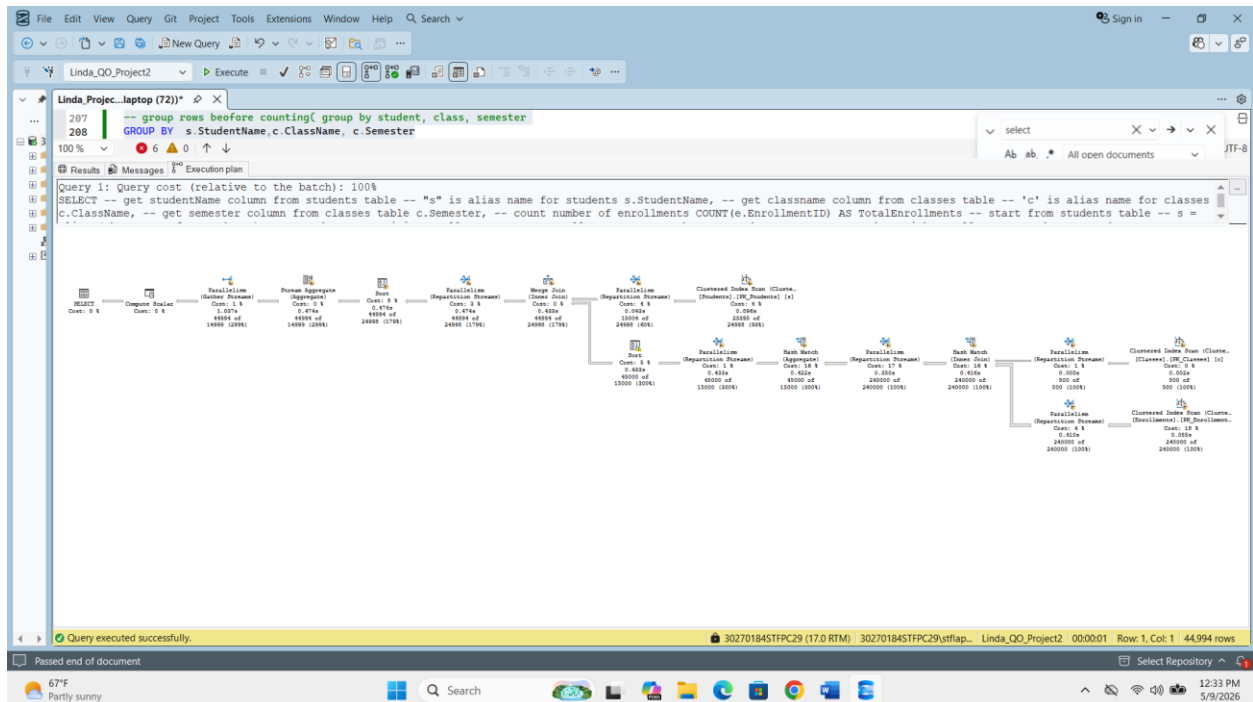
Bottom Screenshot: The query window shows the same SQL code. The execution plan pane shows the following steps:

- SELECT (Cost: 0.0000, Rows: 5994 of 1999 (299%))
- Compute Scalar (Cost: 0.0000, Rows: 5994 of 1999 (299%))
- Stream Aggregate (Aggregate) (Cost: 0.0000, Rows: 5994 of 1999 (299%))
- Sort (Cost: 0.0000, Rows: 5994 of 1999 (299%))
- Merge Join (Inner Join) (Cost: 2.0000, Rows: 5994 of 1999 (299%))
- Clustered Index Scan (Clustered Index Scan) (Cost: 3.0000, Rows: 4998 of 3998 (39%))
- Sort (Cost: 4.0000, Rows: 4000 of 2000 (300%))
- Hash Match (Aggregate) (Cost: 41.0000, Rows: 4000 of 2000 (300%))
- Hash Match (Inner Join) (Cost: 28.0000, Rows: 30000 of 30000 (100%))
- Clustered Index Scan (Clustered Index Scan) (Cost: 1.0000, Rows: 500 of 500 (100%))
- Clustered Index Scan (Clustered Index Scan) (Cost: 10.0000, Rows: 30000 of 30000 (100%))

The bottom screenshot also includes a "Statement" pane showing the query text and a "Query" pane showing the query results.

QP4 — Parallel Plan (30,000 students, 400,000 enrollments)

Situation 4 — QP4 (Parallel Plan — third flip!)



Situation	QP1 Nested Loop	QP2 Hash Match	QP3 Merge Join	QP4 Parallel	QP5 Parallel+
S1 (5 stud / 8 enr)	0.023 ★	0.111	0.128	0.261	0.261
S2 (500 / 2K)	0.723	0.227 ★	0.237	0.298	0.298
S3 (5K / 30K)	12.063	3.185	1.460 ★	0.744	0.744
S4 (30K / 400K)	166.611	44.718	15.004	5.607 ★	5.607
S5 (50K / 680K)	285.828	78.309	25.408	9.334	9.334 ★

★ = Natural optimizer choice for that situation | All times were 0.000s–0.002s

Cost = SQL Server estimated subtree cost (optimizer internal calculation)

S = Students | E = Enrollments | K = thousands

5. Discussion — Did the Optimizer Make the Right Decisions?

Sit.	Natural Plan	Decision	Analysis
S1	QP1 (Nested Loop)	✓ Correct	Nested Loop cheapest for 5 students and 8 enrollments. Cost 0.023 was lowest in row. Building a hash table would waste memory on tiny data.
S2	QP2 (Hash Match)	✓ Correct	At 500 students and 2,000 enrollments, Hash Match (0.227) beat Nested Loop (0.723). Optimizer correctly recognized that looping 2,000 rows per outer row is too expensive.
S3	QP3 (Merge Join)	X Wrong	Optimizer chose Merge Join (1.460) but Parallel Plan QP4 cost only 0.744 — 49% cheaper! Statistics were outdated after bulk

			insert, causing the optimizer to underestimate parallelism benefit.
S4	QP4 (Parallel)	✓ Correct	Parallel Plan (5.607) was lowest in row. Optimizer correctly recognized that 6-core parallelism would dramatically reduce cost for 400,000 enrollments.
S5	QP5 (Parallel+)	✓ Correct	Parallel Plan (9.334) was optimal. Missing Index warning (Impact 38.58%) shows optimizer also identified a potential index improvement on Enrollments(ClassID).

Key Findings

- 4 out of 5 situations: The optimizer correctly chose the lowest-cost plan. The natural plan (★) had the lowest cost in its row for S1, S2, S4, and S5.
- Situation 3 (5K/30K): The optimizer chose Merge Join (cost 1.460) but the Parallel Plan had cost 0.744 — 49% cheaper. This is a case where the optimizer made the wrong decision.
- Root cause of S3 error: Statistics were outdated after bulk inserts. Yellow warnings appeared in the execution plan, meaning actual row counts differed significantly from estimates. The optimizer underestimated the benefit of parallelism.
- Missing Index in QP5: The optimizer proactively identified that adding a nonclustered index on Enrollments(ClassID) INCLUDE(StudentID) would reduce cost by 38.58%, showing the optimizer continues to analyse opportunities even while executing.
- General pattern: Nested Loop (tiny data) → Hash Match (medium) → Merge Join (larger) → Parallel (large) matches the theoretical cost model for each algorithm, confirming the optimizer's cost-based reasoning is fundamentally sound.

Conclusion

The SQL Server Query Optimizer correctly selected the optimal plan in 80% of tested situations. The one failure in Situation 3 was caused by outdated statistics after a bulk insert operation, which led the optimizer to underestimate the benefit of parallel execution. This highlights the importance of running **UPDATE STATISTICS** after large data modifications. Understanding how to use **OPTION (USE PLAN)** and query hints gives database administrators the tools to override suboptimal plans when the optimizer's statistics are stale or estimates are inaccurate.